FPGA Market Maker: Design Document

Ulysses Butler

Fall 2026

Contents

1 The Environment **1**
- 1.1 The simulator . . . 2
- 1.2 From RTL to C++ . . . 2
- 1.3 Driving the FPGA . . . 2
- 1.4 Running it . . . 3

2 The Pipeline **3**
- 2.1 Market data parser (in progress) . . . 5
- 2.2 Sequence check (in progress) . . . 5
- 2.3 Execution report parser . . . 5
- 2.4 Merge . . . 6
- 2.5 Order table . . . 6
- 2.6 Book (in progress) . . . 6
- 2.7 Pricing . . . 7
- 2.8 Quote manager . . . 7
- 2.9 Order encoder (in progress) . . . 8
- 2.10 Register bank and the cold path . . . 8

Appendices **8**

Appendix A Wire Format **8**

Appendix B Market Data API **9**

Appendix C Order API **10**

1 The Environment

The market maker is a hardware design, written in SystemVerilog in `rtl/`. There is no FPGA board in this assignment: the design runs in simulation, connected to a simulated exchange, and everything around it is C++. You won't need to change any of the C++, but this section explains what it does, so you know what is driving your design.

1.1 The simulator

The simulator (`sim/`) is the same one used in the previous assignment: a discrete-event simulation with an exchange at its heart. Other market participants, scripted by a scenario file (`sim/fixtures/*.scn`), send the exchange orders; the exchange matches them, publishes market data (Appendix B), and responds to each participant about its own orders (Appendix C). Our orders and the exchange's messages to us travel over a simulated network with a fixed latency: by default, market data takes 5 µs to reach us, and orders and responses take 10 µs each way.

The application sees the simulator only through three handlers: `OnMarketData` when a market data message arrives, `OnExecutionReport` when the exchange responds about one of our orders, and `OnTimer` when a periodic timer fires. It sends orders with `OrderEntrySession::Send`, from inside one of these handlers.

Time in the simulator is *simulated time*, the scenario's schedule, and it stands still while a handler runs. So how long our code takes never changes which orders we send, or when. In the previous assignment, performance was measured in wall-clock time instead. Here, it is measured in clock cycles of the design, which, multiplied by the clock period `make timing` reports, give the time the FPGA would take.

1.2 From RTL to C++

Verilator compiles the design into a C++ class, `Vmarket_maker`, with one member for each of the top module's ports (`rtl/market_maker.sv`). To simulate a clock cycle, the C++ code sets the inputs, sets `clk` to 0 and calls `eval()`, then sets it to 1 and calls `eval()` again: every register in the design loads its next value on that rising edge, exactly as on a real chip. Verilator checks the design strictly as it compiles it (warnings are errors), so many mistakes, such as a signal that is never used or a latch, stop the build.

1.3 Driving the FPGA

Two small C++ classes in `app/` connect the simulator to the design.

The board (`app/fpga.h`). `Fpga` plays the part of the board around the chip: the network interfaces and the PCIe connection to the CPU. It owns the `Vmarket_maker` and drives its clock. On a real board, messages arrive whenever the network delivers them while the clock runs on its own. Here, the simulator hands over one message at a time, and simulated time stands still until it is handled, so the board processes each message to completion before the next:

1. It copies the message onto the matching 512-bit input (market data or execution reports) and sets that input's valid signal for one cycle.
2. It runs the clock until the design has been quiet (no order out) for 64 cycles, far longer than any message takes to get through.
3. It collects every order that came out, in order, with the number of cycles since the message went in.

Register writes work the same way: the write takes one cycle, then the clock runs until quiet, so the cancels a halt causes come out of the write. Register reads take no cycles.

One consequence: messages never arrive faster than the design handles them, so the simulation never exercises the merge's buffers under load.

The application (`app/market_maker.h`). `MarketMaker` implements the simulator's three handlers. `OnMarketData` and `OnExecutionReport` pass the message's bytes to the board, unread, and send whatever orders come back through `OrderEntrySession::Send`. The CPU makes no decisions on the hot path. `OnTimer` is the cold path: it reads the mid, position, and cash registers, and writes the credit and halt registers (Section 2.10). Its parameters (the credit multiplier, the volatility decay, and the loss limit) are in `app/config.h`; the parameters the FPGA uses (the quote size and the position limit) are in `rtl/config_pkg.sv`.

1.4 Running it

`make tools` Downloads the FPGA tools (Verilator, Yosys, nextpnr, and GTKWave) into `tools/`. Run it once.

`make` Builds `build/market_maker`: the design, compiled by Verilator, with the simulator and the code in `app/`.

`make verify` Runs every scenario in `sim/fixtures/` and checks that the orders sent are exactly the expected ones.

`make test-holes` Builds and runs the unit tests in `tests/`, one per module you implement. These are faster and more isolated than `make verify`: a good first check while you're still working on a single module.

`make timing` Synthesizes, places, and routes the design for a real FPGA, and reports its maximum clock frequency, overall and module by module (a few minutes).

To run one scenario, use `build/market_maker sim/fixtures/basic.scn`. It writes every order sent to `build/out/basic/orders.txt`, and prints how many clock cycles each order took, from the message that caused it. Two options help with debugging: `--trace` writes every message, in both directions, to `trace.txt`, and `--waves` records every signal in the design, every cycle, to `waves.vcd`, which you can open in GTKWave (`tools/oss-cad-suite/bin/gtkwave`) to watch an item move through the pipeline.

2 The Pipeline

The market maker's hot path runs entirely on the FPGA, as a pipeline of modules in `rtl/`. Every message from the exchange, market data or execution report, goes in at the top, and any orders it causes come out at the bottom. The CPU never sees a message on the way; it only adjusts the pipeline's parameters through a small set of registers, once per timer tick (Section 2.10). Figure 1 shows the modules and what flows between them.

Clock and registers. The whole design runs on one clock. Every module registers its outputs, so a signal never has to get through more than one module's logic in a single clock cycle, and the slowest module sets the clock speed for all of them (`make timing` reports each module's maximum clock frequency). Each output comes with a *valid* signal that is high in exactly the cycles the output carries something.

Items. After the merge, everything travels as an *item*: one market data event or one execution report, plus whatever the stages before have added to it. Each stage passes on what later stages need and drops what they don't, so the item's type changes as it goes (`pipeline_item_t`, then `book_item_t`, and so on). An execution report travels all the way to the quote manager; a market data event is turned into changes to the book by the order table and goes no further.

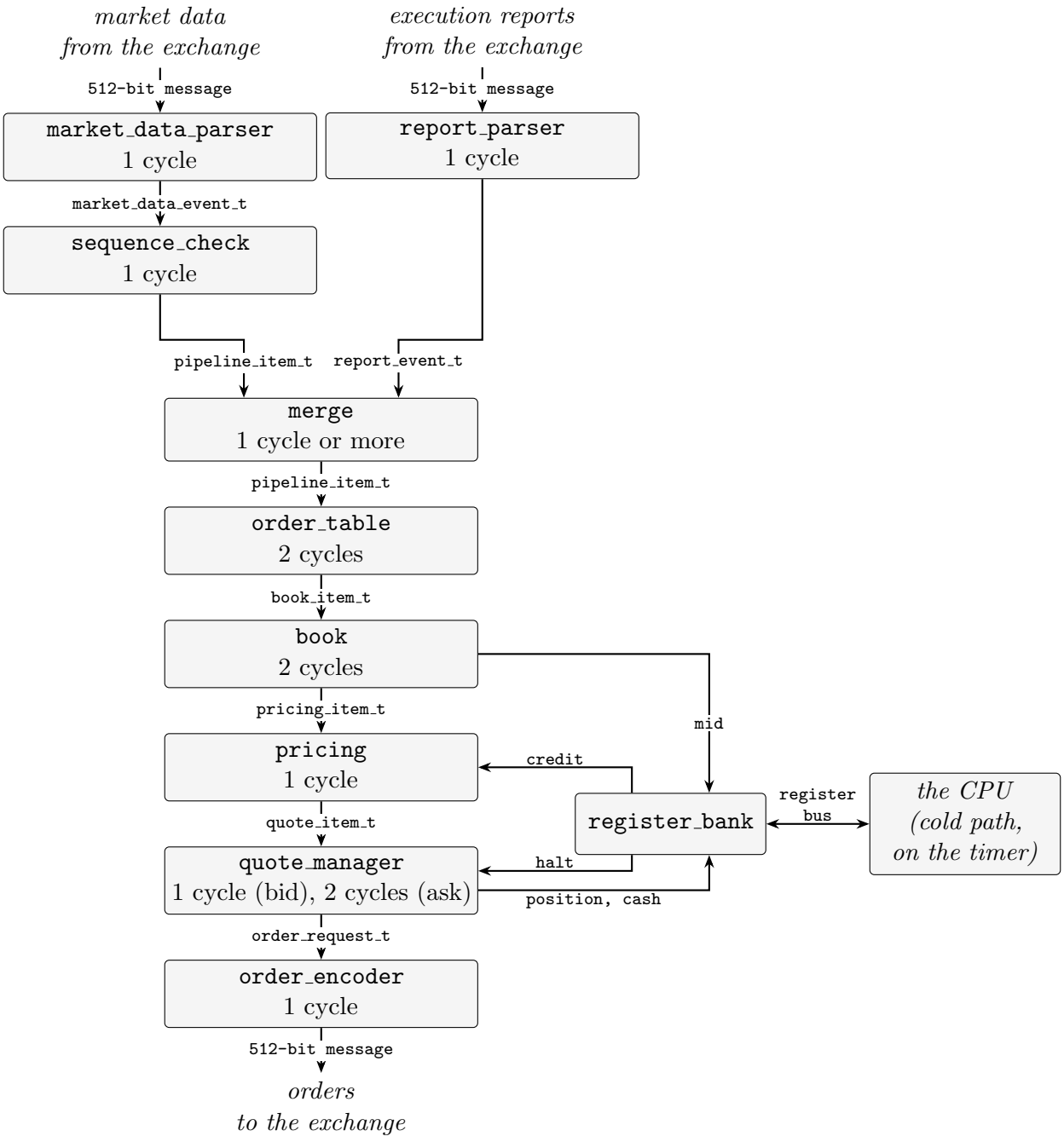


Figure 1: The pipeline. Each box gives the module's name (its file in `rtl/` is the name plus `.sv`) and its latency: how many clock cycles after an item enters it leaves. Each arrow is labeled with the type that travels on it, defined in `rtl/messages_pkg.sv` and `rtl/pipeline_pkg.sv`.

Error flags. Every item carries a set of error flags, each set by the stage that can detect the problem and acted on by the quote manager. Three of them are *sticky*: once set, they stay set on every item until reset, because after them the book can no longer be trusted. The fourth applies to one item only.

Flag	Set by	Meaning
`sequence_gap` (sticky)	sequence check	A market data message was missed.
`unknown_order` (sticky)	order table	The feed referred to an order the table doesn't hold.
`book_overflow` (sticky)	book	A side needed more than 16 price levels.
`book_invalid`	book	After this item, a side is empty or the book is crossed.

Rate. A market data message and an execution report may arrive in the same cycle, and either stream may deliver a message every cycle up to the merge. After the merge, an item leaves at most once every two cycles (an *initiation interval* of two), because the book takes two cycles to apply some items; the merge buffers whatever has to wait.

Latency. An order leaves the FPGA 9 clock cycles after the market data message that caused it arrives, or 8 after an execution report (reports skip the sequence check).

2.1 Market data parser (in progress)

In progress. *This module is one of the four you implement. The description below is what it must do once you're done, not a description of code that's already there.*

The market data parser decodes one whole market data message per cycle into an event, a `market_data_event_t`. It reads the type byte, then picks each field out of the message at its fixed offset (Appendix B): the sequence number, the order ref, and, depending on the type, the new order ref, the side, the shares, and the price. Fields a message doesn't have are zero in the event. The instrument, the timestamp, and the match number are not decoded: nothing downstream uses them. A message of an unknown type produces no event.

2.2 Sequence check (in progress)

In progress. *This module is one of the four you implement. The description below is what it must do once you're done, not a description of code that's already there.*

The sequence check detects missed market data messages. The feed numbers its messages 1, 2, 3, and so on, so the check keeps the number it expects next, starting at 1. If a message carries any other number, at least one message was missed, and it sets `sequence_gap` on that item and every item after it. Either way, the expected number becomes the message's number plus one. It also wraps each event into the first `pipeline_item_t`: the event, the error flags, and a field saying the item came from market data.

2.3 Execution report parser

The execution report parser does for execution reports what the market data parser does for market data, producing a `report_event_t` (Appendix C). It decodes the token, the side, the shares, the price, and the order ref, and for a `Replaced` report, the previous token too. The *shares* field means different things in different reports: an order's size in `Accepted` and `Replaced`, the shares that

traded in `Executed`, and the shares removed in `Canceled`. Reports have no gap check: they travel over a reliable connection, unlike market data.

2.4 Merge

The merge combines the two streams into one, so that every later stage sees market data and reports in a single, well-defined order. It sends at most one item every two cycles. When an item of each kind is waiting, market data goes first. Anything that can't leave yet waits in a one-entry buffer for its stream.

One entry per stream is enough because of the network: the shortest message is 26 bytes and a 64-bit link carries 8 bytes per cycle, so neither stream can deliver more than one message every 4 cycles. That is at most 2 items every 4 cycles, exactly what the merge can send. If messages ever arrive faster than that, an item is lost, and the merge raises its sticky `overflow` output (the simulator then stops with an error).

2.5 Order table

Most market data messages name an order only by its ref ("order 1234 lost 50 shares"), but the book is organized by price. The order table remembers every live order's side, price, and remaining shares, and turns each market data event into one or two *level updates*: add or remove some number of shares at one price on one side. An `OrderReplace` needs two: remove the old order's shares, and add the new order's.

The table also tracks which orders are ours, because the book must count only other participants' orders. An `Accepted` or `Replaced` report tells it that an order ref is ours. The feed usually announces the order first (market data is faster than our acknowledgments); then the order's remaining shares are removed again. If the report arrives first, the order is marked as ours before the feed's `AddOrder`, which then adds nothing. Either way, an order's shares are in the book exactly while it is live and not ours.

The table lives in block RAM, with 4,096 entries chosen by the low 12 bits of the order ref. Each entry also stores the rest of the ref, so a newer order that lands on the same entry can't be mistaken for an older one. But each ref has only one entry, so an order that rests while 4,096 newer orders arrive loses its entry. If the feed later refers to it, the table can't find it, and sets `unknown_order`.

The table takes two cycles: one to read the order's entry, one to update it.

2.6 Book (in progress)

In progress. *This module is one of the four you implement. The description below is what it must do once you're done, not a description of code that's already there.*

The book keeps the best 16 price levels of each side (excluding our orders), as a list in registers sorted best first: bids highest first, asks lowest first. Each level holds a price and the total shares at that price.

A level update compares its price against all 16 levels at once, in parallel. Then, in a single cycle, it either changes the size of the matching level, removes that level (if its size drops to zero, moving every worse level up one place), or inserts a new level (moving every worse level down one place). Reading the top of book is free: it is the first level of each side.

An item with two level updates takes two cycles, one update per cycle; doing both in one cycle would need the compare-and-move logic twice in a row, several times larger and slower. This is why the pipeline can accept an item only every two cycles.

If a side would need a 17th level, the book can't be trusted any more (once the better levels empty, the dropped one should become visible), so it sets `book_overflow`. After every item, it sets `book_invalid` if a side is empty or the book is crossed, and passes the top of book on to pricing.

2.7 Pricing

Pricing turns the top of book into the bid and offer we want to quote, using the microprice as fair value:

$$\text{fair value} = \frac{B \cdot \text{ask size} + A \cdot \text{bid size}}{\text{bid size} + \text{ask size}},$$

where B and A are the best bid and offer. A divider is far too large and slow for the hot path, but we never need fair value itself: our quotes are on the tick grid, so we only need the ticks just below and above it. Fair value always lies between B and A, and

$$\text{fair value} - B = (A - B) \cdot \frac{\text{bid size}}{\text{bid size} + \text{ask size}},$$

so if the spread is S ticks, the tick just below fair value is $B + k$ ticks, where k is the largest whole number with

$$k \cdot (\text{bid size} + \text{ask size}) \le S \cdot \text{bid size}.$$

Since k is between 0 and $S - 1$, pricing checks every candidate at once, with one comparison per possible k, and takes the largest that holds. Then

$$\text{bid} = (\text{tick at or below fair value}) - \text{credit},$$
$$\text{ask} = (\text{tick at or above fair value}) + \text{credit},$$

where the credit, a whole number of ticks, comes from the register bank.

One comparison per possible k means the hardware can only handle spreads up to some size, here 16 ticks; wider markets aren't quoted. Pricing also doesn't quote when the book is invalid, or when the bid would not be positive.

2.8 Quote manager

The quote manager owns our two quotes, one bid and one offer, and decides after every item whether to change them. For each side it tracks the resting order (its token, price, and open shares) and any request still in flight; it also tracks our position and cash.

For each item, it:

1. **Applies the report**, if the item is one. An `Accepted` or `Replaced` turns the side's request in flight into its resting order. An `Executed` changes our position and cash, and the order's open shares. A `Rejected` or `Canceled` clears the request or order it names.
2. **Decides.** On any sticky error flag, or when the CPU has set the halt register, it halts. Otherwise, if pricing could quote, each side with no request in flight gets a new order (if it has none) or a replacement (if its price is no longer the one pricing wants), unless the position limit forbids it: no buy that could take our position above +500 shares, and no sell below −500.
3. **Sends** the requests to the order encoder, with tokens numbered 1, 2, 3, and so on.

Both sides may need an order for the same item, but the encoder takes one request per cycle. Items arrive at least two cycles apart, so the bid's request leaves in the item's first cycle and the ask's in the second.

A halt is permanent. A small state machine then cancels everything we have at the exchange, one cancel per cycle: each side's resting order and any request still in flight. Nothing is quoted again, but reports keep updating our position and cash.

2.9 Order encoder (in progress)

In progress. *This module is one of the four you implement. The description below is what it must do once you're done, not a description of code that's already there.*

The order encoder is the market data parser in reverse: it turns one `order_request_t` per cycle into a whole `EnterOrder`, `ReplaceOrder`, or `CancelOrder` message (Appendix C), writing each field at its fixed offset. Every order is a `Day` order for the one instrument.

2.10 Register bank and the cold path

The register bank is the CPU's only way into the FPGA. On a real board it would sit behind PCIe, where the CPU's stores to an address become writes here, and its loads become reads. Here, the C++ code in `app/` drives the same bus directly.

Address	Register	Access	Meaning
0	credit	write	Half-spread, in price units; a whole number of ticks.
1	halt	write	Nonzero: cancel everything and stop quoting.
2	position	read	Shares, signed (positive is long).
3	cash	read	Price units times shares, signed.
4	mid	read	Bit 63: the book is valid; low 32 bits: $(B + A)/2$.

The CPU's *cold path* runs on every timer tick. It reads the mid and, if the book is valid, updates its estimate of volatility; it then writes the credit that volatility implies, rounded up to a whole number of ticks. It also marks our position to the mid, and writes the halt register if the loss is over the limit. Nothing on the cold path is time-critical, so it uses floating point and runs on the CPU.

Appendix A Wire Format

Every message, in either direction, is a packed sequence of fixed-size fields: no padding, no variable-length fields, and no message longer than 64 bytes. The first byte is always the message type, an ASCII character. Multi-byte integers are little-endian: the least significant byte comes first.

The field types are:

`char` One byte holding an ASCII character, used for the message type and other small enumerations (for example `B` for buy and `S` for sell).

`u16`, `u32`, `u64` Unsigned integers of 2, 4, and 8 bytes.

`price` A signed 8-byte integer in fixed point with 4 implied decimal places: \$100.25 is 1 002 500. Valid prices are positive multiples of the tick size, \$0.01 (100).

On the FPGA, each message arrives whole, in a single clock cycle, on a 512-bit (64-byte) bus. Byte 0 is in bits `[7:0]`, byte 1 in bits `[15:8]`, and so on; bytes past the end of the message are zero. So a field of n bytes at offset o is the bit slice `data[8*o +: 8*n]`, already in the right byte order.

Appendix B Market Data API

The market data feed is modeled on NASDAQ ITCH. It is order-by-order: rather than publishing price levels, it reports every individual order that is added to or removed from the book. Every order is displayed, so the feed includes the application's own orders, unmarked.

Every message starts with a 19-byte header carrying the message type, the instrument, a **sequence number** (starting at 1 and increasing by 1 per message, so a gap means a message was missed), and the time the exchange processed the event.

Offset	Size	Field	Type
0	1	message type	`char`
1	2	instrument	`u16`
3	8	sequence	`u64`
11	8	timestamp	`u64`, nanoseconds

`AddOrder` (`A`, 40 bytes)

Fields: order ref, side, shares, price.

A new order is resting on the book. The order ref is the exchange's identifier for it; later messages about this order carry only the ref.

Offset	Size	Field	Type
0	19	header	
19	8	order ref	`u64`
27	1	side	`char`: `B` buy, `S` sell
28	4	shares	`u32`
32	8	price	`price`

`OrderExecuted` (`E`, 39 bytes)

Fields: order ref, executed shares, match number.

Part or all of a resting order traded. This is the feed's trade print.

Offset	Size	Field	Type
0	19	header	
19	8	order ref	`u64`
27	4	executed shares	`u32`
31	8	match number	`u64`

`OrderCancel` (`X`, 31 bytes)

Fields: order ref, canceled shares.

A resting order was reduced in size, but is still on the book.

Offset	Size	Field	Type
0	19	header	
19	8	order ref	`u64`
27	4	canceled shares	`u32`

`OrderDelete` (`D`, **27 bytes**)

Fields: order ref.

A resting order was removed from the book entirely.

Offset	Size	Field	Type
0	19	header	
19	8	order ref	`u64`

`OrderReplace` (`U`, **47 bytes**)

Fields: original order ref, new order ref, shares, price.

A resting order was replaced by a new one at a new price and/or size. The new order has a new ref and loses its place in the queue.

Offset	Size	Field	Type
0	19	header	
19	8	original order ref	`u64`
27	8	new order ref	`u64`
35	4	shares	`u32`
39	8	price	`price`

Appendix C Order API

The order entry protocol is modeled on NASDAQ OUCH. The application identifies its own orders by a **token** that it chooses, which must be unique for the whole session. The exchange's order ref is only known once the exchange's response arrives, a full round trip later.

Requests (application to exchange). Requests have no header beyond the message type.

`EnterOrder` (`O`, **25 bytes**)

Fields: token, side, shares, instrument, price, time in force (`Day` or `IOC`).

Places a new order. A `Day` order rests on the book until it is filled or canceled; an `IOC` (immediate-or-cancel) order trades what it can immediately and cancels the rest. A reused token is silently ignored, so a retransmission is safe.

Offset	Size	Field	Type
0	1	message type	`char`: `O`
1	8	token	`u64`
9	1	side	`char`: `B` buy, `S` sell
10	4	shares	`u32`
14	2	instrument	`u16`
16	8	price	`price`
24	1	time in force	`char`: `D` day, `I` IOC

`ReplaceOrder` (`U`, **29 bytes**)

Fields: existing token, replacement token, shares, price.

Replaces a live order with a new one at a new price and/or size. The replacement takes the new token and loses its place in the queue.

Offset	Size	Field	Type
0	1	message type	`char`: `U`
1	8	existing token	`u64`
9	8	replacement token	`u64`
17	4	shares	`u32`
21	8	price	`price`

`CancelOrder` (`X`, **13 bytes**)

Fields: token, shares.

Reduces a live order to `shares` remaining; 0 cancels it entirely. Requests for unknown or already-dead tokens are ignored.

Offset	Size	Field	Type
0	1	message type	`char`: `X`
1	8	token	`u64`
9	4	shares	`u32`, shares remaining

Responses (exchange to application). Each response carries a 17-byte header with its type, a sequence number (per session, starting at 1), and the time the exchange processed the event.

Offset	Size	Field	Type
0	1	message type	`char`
1	8	sequence	`u64`
9	8	timestamp	`u64`, nanoseconds

`Accepted` (`A`, **49 bytes**)

Fields: token, side, shares, instrument, price, time in force, order ref.

The order was accepted. This is the first time the application learns its order ref.

Offset	Size	Field	Type
0	17	header	
17	8	token	`u64`
25	1	side	`char`: `B` buy, `S` sell
26	4	shares	`u32`
30	2	instrument	`u16`
32	8	price	`price`
40	1	time in force	`char`: `D` day, `I` IOC
41	8	order ref	`u64`

`Replaced` (`U`, **54 bytes**)

Fields: replacement token, previous token, side, shares, price, order ref.

The replacement is live, with a new order ref.

Offset	Size	Field	Type
0	17	header	
17	8	replacement token	`u64`
25	8	previous token	`u64`
33	1	side	`char`: `B` buy, `S` sell
34	4	shares	`u32`
38	8	price	`price`
46	8	order ref	`u64`

`Executed` (`E`, 46 bytes)

Fields: token, executed shares, price, match number, liquidity flag.

One of our orders traded. The liquidity flag says whether our order was resting (added liquidity) or was the aggressor (removed liquidity).

Offset	Size	Field	Type
0	17	header	
17	8	token	`u64`
25	4	executed shares	`u32`
29	8	price	`price`
37	8	match number	`u64`
45	1	liquidity flag	`char`: `A` added, `R` removed

`Rejected` (`J`, 26 bytes)

Fields: token, reason.

The request was refused: an invalid price or size, an unknown instrument, or a replace of an order that is no longer live.

Offset	Size	Field	Type
0	17	header	
17	8	token	`u64`
25	1	reason	`char`: `X` price, `Z` shares, `S` instrument, `D` unknown order

`Canceled` (`C`, 30 bytes)

Fields: token, shares canceled, reason.

Some or all of an order was canceled, either at our request or because it was the unfilled remainder of an `IOC` order.

Offset	Size	Field	Type
0	17	header	
17	8	token	`u64`
25	4	shares canceled	`u32`
29	1	reason	`char`: `U` requested, `I` IOC remainder